SimpleFS — Line-by-Line Code Explanation & Viva Preparation Guide

CSE321 — Operating Systems, Summer 2026

Lab Term Project: SimpleFS (Simple File System in C)

Section 07 — Group 14

Afia Tasnim (23301678) • Md Rakib Hasan (23201141)

This document explains every line of simplefs.h, simplefs_builder.c, and simplefs_adder.c in plain language, so either group member can confidently answer any viva question about any part of the code — not just the part they typed.

How to use this document

Part 1 explains the big-picture concepts in plain words. Part 2 goes through simplefs.h. Part 3 goes through simplefs_builder.c (primarily built and tested by **Afia**, but explained here for both members). Part 4 goes through simplefs_adder.c (primarily built and tested by **Rakib**, explained here for both members). Part 5 is a bank of likely viva questions with model answers, including tricky 'what if' questions examiners like to ask. Both members should read all of it — the viva is individual, and the instructor may ask either of you about either file.

PART 1 — The Big Picture (Read This First)

1.1 What is a file system, really?

A hard disk is just a huge flat array of bytes. It doesn't know what a 'file' is. A file system is a set of rules and data structures layered on top of that raw array of bytes so that we can organize it into files and folders. SimpleFS is a toy version of this: instead of a real disk, we use one ordinary binary file called disk.img (262144 bytes = 256 KiB), and instead of a real OS mounting it, our two C programs read and write directly into specific byte offsets inside that file using fseek() and fread()/fwrite().

1.2 Blocks — cutting the disk into fixed-size pieces

SimpleFS divides the 262144-byte image into 64 equal chunks called **blocks**, each 4096 bytes. Block number 0 starts at byte 0, block 1 starts at byte 4096, block 2 starts at byte 8192, and so on. In general:

```
starting_byte_of_block_N = N * BLOCK_SIZE (N * 4096)
```

That's the single most important formula in the whole project — every fseek() call in the code is really just this formula in disguise.

1.3 The five regions of the disk

Region	Block(s)	What it stores
Superblock	0	'ID card' of the file system — sizes, magic number, where everything else lives
Inode Bitmap	1	One bit per inode (32 bits used) — is this inode free or taken?
Data Bitmap	2	One bit per data block (60 bits used) — is this block free or taken?
Inode Table	3	32 inode records back to back, 128 bytes each = exactly one block
Data Region	4–63	The actual file contents and the root directory's entries live here

1.4 The superblock

Think of the superblock as the label on a filing cabinet that tells you how the cabinet is organized before you open any drawer. It stores things like the magic number (a fixed signature 0x53465331 used to confirm 'yes, this file really is a SimpleFS image and not garbage'), the block size, how many blocks/inodes exist, and which block each region starts at.

1.5 Inodes — the file's ID card

An **inode** ('index node') stores everything about a file EXCEPT its name: type (file or directory), size in bytes, and up to 3 **direct pointers** — the block numbers where the file's actual data is stored. Notice the inode does

NOT know its own file name — that mapping lives only in the directory entry. This is a classic real-world file-system idea (it's why you can hard-link one inode to multiple names in real Unix file systems, although SimpleFS doesn't implement links).

1.6 Bitmaps — a yes/no checklist using single bits

A bitmap is just an array of bytes where we borrow individual **bits** as tiny 0/1 flags to avoid wasting a whole byte per flag. Bit value 0 = free/unused, 1 = allocated/used. We have one bitmap for inodes (32 bits needed, block 1) and one for data blocks (60 bits needed, block 2). Even though 32 bits only needs 4 bytes, the spec still reserves the FULL 4096-byte block for each bitmap — the extra bytes just sit there as zero, unused padding, so the layout stays clean and block-aligned.

To set bit number *index* to 1 (mark allocated):

```
bitmap[index / 8] |= (1 << (index % 8));
```

Why divide by 8? Because each byte holds 8 bits. index/8 tells you WHICH BYTE the bit lives in. index%8 tells you WHICH POSITION inside that byte (0–7). 1 << (index%8) creates a byte that is all zero except a single 1 in that position (e.g. index%8==3 gives 00001000). The |= (bitwise OR-and-assign) turns that one bit on without disturbing any of the other 7 bits already in that byte. Checking a bit uses & (bitwise AND) instead of |= — AND-ing with a mask that has only one bit set gives a non-zero result only if that specific bit is 1.

1.7 Directory entries — mapping a name to an inode

A **directory entry (dirent)** is the missing link: it says 'the name “test1.txt” refers to inode number 2'. SimpleFS only has ONE directory — the root ("/") — and it lives in data block 4. That block is big enough to hold 4096/64 = 64 entries. The first two entries (index 0 and 1) are always the special entries "." and "..", both of which point back at inode 1 (the root itself, since root has no parent). That leaves 62 slots, but since SimpleFS only has 32 inodes total and inode 1 is reserved for root, the real maximum is 31 user files.

1.8 First-fit allocation

Whenever we need a free inode or a free data block, we scan the corresponding bitmap from the very beginning and grab the FIRST bit that is still 0. This strategy is called **first-fit**. It's simple (a straight linear scan) and it's exactly what the spec requires and what the grader will check for.

1.9 The three kinds of numbers you must never confuse

This is the #1 source of bugs in this project, and a favourite viva trap question. There are THREE different numbering systems in play, and they are NOT the same number:

Concept	Starts at	Example	Converts to absolute block via
Inode number	1	Inode 2	table index = inode_number – 1
Inode table index	0	Index 1	byte offset = 12288 + index × 128
Data bitmap index	0	Index 1	absolute block = 4 + index
Absolute block number	0	Block 5	byte offset = block_number × 4096

Example: data bitmap index 1 is NOT block 1 — it's absolute block 5 (4 + 1), because the data region starts at block 4. Mixing these up is the single most common mistake in this project (the spec even lists it first under 'Common Mistakes').

PART 2 — simplefs.h Explained Line by Line

BOTH MEMBERS — shared file

This header file was provided by the instructor and must NOT be modified. Both of you must still understand every line, because every constant used in both .c files is defined here.

2.1 Include guard

```
#ifndef SIMPLEFS_H
#define SIMPLEFS_H
...
#endif
```

Both simplefs_builder.c and simplefs_adder.c do #include "simplefs.h". If a header gets included twice in the same compilation (directly or indirectly), the compiler would see the same struct defined twice and error out. The include guard says: 'if SIMPLEFS_H hasn't been defined yet, define it now and process the file; if it's already been defined, skip everything until #endif'. This is a standard C idiom used in essentially every header file.

2.2 The #include lines

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
```

stdio.h gives us printf, fopen, fread, fwrite, fseek, fclose — everything we need to talk to the disk.img file. **stdint.h** gives fixed-width integer types such as uint32_t, which is guaranteed to be exactly 4 bytes on every platform. This matters enormously for a binary file format: if we used plain 'int', its size could technically differ across compilers/platforms, which would silently corrupt our fixed 128-byte inode / 64-byte dirent layout. **stdlib.h** is the general utility header. **string.h** gives strcmp (compare names), strncpy (copy a name safely), strlen (get name length), and memset (zero out a block of memory).

2.3 The #define constants

BLOCK_SIZE 4096 — Every block is 4096 bytes.

TOTAL_BLOCKS 64 — The whole image has 64 blocks -> 64*4096 = 262144 bytes.

TOTAL_INODES 32 — There are 32 inode 'slots' in the inode table.

SUPERBLOCK_BLOCK 0 — The superblock lives at block index 0.

INODE_BITMAP_BLOCK 1 — The inode bitmap lives at block index 1.

DATA_BITMAP_BLOCK 2 — The data bitmap lives at block index 2.

INODE_TABLE_BLOCK 3 — The inode table lives at block index 3.

DATA_REGION_BLOCK 4 — The first usable data block is block 4 (blocks 0-3 are reserved for metadata).

DATA_BLOCKS 60 — 64 total blocks - 4 reserved = 60 blocks available for data.

ROOT_INODE 1 — The root directory always uses inode number 1.

ROOT_DATA_BLOCK 4 — The root directory's entries are stored starting at block 4.

MAGIC_NUMBER 0x53465331 — A fixed 4-byte 'signature'. When simplefs_adder opens an image, it checks this value to make sure it's really a SimpleFS image and not a random/corrupted file.

TYPE_FILE 1 / TYPE_DIRECTORY 2 — Small integer tags used in both inode.type and dirent.type to say what kind of object this is.

MAX_DIRECT_BLOCKS 3 — Every inode has exactly 3 direct block pointers -> direct[0..2].

MAX_FILE_SIZE (3*4096=12288) — The biggest a file can be, since we only have 3 direct pointers and no indirect blocks.

DIRENT_SIZE 64 — Size in bytes of one directory entry record.

2.4 The three structs

superblock_t (36 bytes of real data, sitting inside a 4096-byte block):

```
typedef struct {
uint32_t magic, block_size, total_blocks, inode_count;
uint32_t inode_bitmap_block, data_bitmap_block,
inode_table_block, data_region_block;
uint32_t root_inode;
} superblock_t;
```

9 fields, each a uint32_t (4 bytes) = 36 bytes total. magic is the signature check. block_size/total_blocks/inode_count describe the overall geometry. The next four fields tell you WHICH block each region starts at (so technically the code doesn't even have to hard-code DATA_REGION_BLOCK — it could read it from the superblock — but our implementation uses the #define constants directly for simplicity, matching what the spec expects). root_inode records that inode 1 is the root.

inode_t (must be exactly 128 bytes):

```
typedef struct {
uint16_t type, links;
uint32_t size;
uint32_t direct[3];
uint8_t reserved[108];
} inode_t;
```

type (2 bytes) + links (2 bytes) + size (4 bytes) + direct[3] (3×4=12 bytes) = 20 bytes of real data. The remaining reserved[108] bytes pad the struct out to exactly 128 bytes and are always kept zero — they exist so the inode table lines up cleanly (32 inodes × 128 bytes = exactly one 4096-byte block). type says file vs directory. links counts how many directory entries point at this inode (root has 2: "." and ".."; a normal file has 1). size is the file's REAL size in bytes (not rounded up to a block boundary). direct[0..2] hold up to 3 absolute block numbers where this file's data physically lives; unused slots stay 0.

dirent_t (must be exactly 64 bytes):

```
typedef struct {
uint32_t inode_no;
uint8_t type;
char name[59];
} dirent_t;
```

inode_no (4 bytes) + type (1 byte) + name[59] (59 bytes) = 64 bytes exactly. inode_no of 0 means 'this slot is empty/unused' (since inode numbers start at 1, 0 is never a valid real inode, so it's a safe 'nothing here' marker). name holds up to 58 real characters plus 1 mandatory null terminator byte '\0', which is why the spec caps file names at 58 characters, not 59.

2.5 Function prototypes

```
void set_bit(unsigned char *bitmap, int index);
int is_bit_set(unsigned char *bitmap, int index);
int find_free_inode(unsigned char *bitmap);
int find_free_data_block(unsigned char *bitmap);
```

```
long inode_offset(int inode_number);
```

These are just declarations ('here is what this function looks like') so both .c files know these functions exist and can call them. The actual code (the function bodies) is written separately inside each .c file — note that both simplefs_builder.c and simplefs_adder.c each define their OWN copies of set_bit, is_bit_set and inode_offset (identical implementations), because each program is compiled and linked completely independently — they never run in the same process.

PART 3 — simplefs_builder.c Explained Line by Line

Afia's part — but Rakib must know this too

Job of this program: create a brand-new 262144-byte disk.img from nothing and set it up as a valid, empty SimpleFS: write the superblock, mark inode 1 and block 4 as used, create the root inode, and write the "." and ".." entries.

3.1 Helper functions at the top

```
void set_bit(unsigned char *bitmap, int index) {
bitmap[index / 8] |= (1u << (index % 8));
}
int is_bit_set(unsigned char *bitmap, int index) {
return bitmap[index / 8] & (1u << (index % 8));
}
long inode_offset(int inode_number) {
return ((long)INODE_TABLE_BLOCK * BLOCK_SIZE)
+ ((long)(inode_number - 1) * sizeof(inode_t));
}
int find_free_inode(unsigned char *bitmap) { (void)bitmap; return -1; }
int find_free_data_block(unsigned char *bitmap) { (void)bitmap; return -1; }
```

set_bit and is_bit_set implement the bit trick explained in Part 1.6. inode_offset(n) converts an inode NUMBER into the exact BYTE POSITION of that inode inside the image file: start of block 3, plus (n-1) inodes of 128 bytes each, because inode 1 sits at table index 0 (n-1 = 0), inode 2 at index 1, etc. find_free_inode and find_free_data_block are just stubs here (always return -1, 'not found') — the builder program never actually needs to search for a free inode/block, because it only ever creates the ONE root inode and ONE root block, both by hand. (void)bitmap; is there purely to silence an 'unused parameter' compiler warning, since these two stub functions never end up used elsewhere in this particular file.

3.2 main() — argument parsing

```
int main(int argc, char *argv[])
{
char *image_name = NULL;
FILE *fp;
unsigned char zero_block[BLOCK_SIZE] = {0};
unsigned char inode_bitmap[BLOCK_SIZE] = {0};
unsigned char data_bitmap[BLOCK_SIZE] = {0};
superblock_t sb;
inode_t root_inode;
dirent_t dot, dotdot;

if (argc != 3) { printf("Usage: %s --image <image_name>\n", argv[0]); return 1; }
if (strcmp(argv[1], "--image") != 0) { printf("Error: expected --image option.\n"); return 1; }
image_name = argv[2];
```

argc is the argument COUNT and argv is the array of argument STRINGS. Running "./simplefs_builder --image disk.img" gives argc = 3 (program name + 2 more words) and argv = {"./simplefs_builder", "--image", "disk.img"}. We check argc != 3 to reject wrong usage (e.g. missing the image name), and strcmp checks that the second word is exactly "--image" (strcmp returns 0 only on an exact match).
zero_block/inode_bitmap/data_bitmap are all declared as BLOCK_SIZE-byte arrays and initialized to {0} — meaning every one of their 4096 bytes starts as zero, which is exactly what we want for 'nothing allocated yet'.

3.3 Creating and zero-filling the image

```
fp = fopen(image_name, "wb+");
if (!fp) { printf("Error: could not create image file.\n"); return 1; }

for (int i = 0; i < TOTAL_BLOCKS; i++) {
if (fwrite(zero_block, BLOCK_SIZE, 1, fp) != 1) {
printf("Error: could not initialize image.\n"); fclose(fp); return 1;
}
}
```

"wb+" opens the file for both reading and writing in BINARY mode, and creates it if it doesn't exist (truncating it to empty if it does). We then write the all-zero zero_block 64 times in a row (TOTAL_BLOCKS = 64), which grows the file from 0 bytes up to the full 262144 bytes, entirely filled with zero for now. This guarantees that every byte we don't explicitly set later — like the padding bytes in the superblock or the unused inode slots — is correctly zero, matching the spec's requirement everywhere it says 'remaining bytes must be zero'. fwrite returns how many complete items it wrote (we asked for 1 item of size BLOCK_SIZE); if it's not exactly 1, something went wrong (e.g. disk full), so we fail cleanly.

3.4 TODO 1 — Filling the superblock

```
memset(&sb, 0, sizeof(sb));
sb.magic = MAGIC_NUMBER;
sb.block_size = BLOCK_SIZE;
sb.total_blocks = TOTAL_BLOCKS;
sb.inode_count = TOTAL_INODES;
sb.inode_bitmap_block = INODE_BITMAP_BLOCK;
sb.data_bitmap_block = DATA_BITMAP_BLOCK;
sb.inode_table_block = INODE_TABLE_BLOCK;
sb.data_region_block = DATA_REGION_BLOCK;
sb.root_inode = ROOT_INODE;
fseek(fp, SUPERBLOCK_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(&sb, sizeof(sb), 1, fp);
```

memset zeroes the whole struct first as a safety habit (good practice before filling a struct that will be written to disk, so there's never leftover garbage memory in it). Then each field is filled with the required value straight from the constants in simplefs.h. fseek(fp, SUPERBLOCK_BLOCK * BLOCK_SIZE, SEEK_SET) moves the file's read/write cursor to byte 0*4096 = byte 0 (SEEK_SET means 'measure from the start of the file'). fwrite(&sb;, sizeof(sb), 1, fp) then writes sizeof(sb) = 36 bytes starting at that position. Only 36 of the 4096 bytes in block 0 get overwritten — the rest were already zero from step 3.3, exactly matching the spec's 'remaining 4060 bytes must be zero' requirement.

3.5 TODO 2 — Marking inode 1 allocated

```
set_bit(inode_bitmap, 0);
fseek(fp, INODE_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(inode_bitmap, BLOCK_SIZE, 1, fp);
```

set_bit(inode_bitmap, 0) turns on bit 0 of the local inode_bitmap array in memory (bit 0 always represents inode 1, since 'inode number = bit index + 1'). This makes byte 0 of that array become 00000001 in binary = 0x01 in hex — exactly the '01' you see when inspecting the image with xxd, matching the spec's Test Case 3. Then we seek to block 1 and write the WHOLE 4096-byte bitmap array to disk (even though only bit 0 is meaningfully set — the rest are already 0 from initialization).

3.6 TODO 3 — Marking the root's data block allocated

```
set_bit(data_bitmap, 0);
fseek(fp, DATA_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(data_bitmap, BLOCK_SIZE, 1, fp);
```

Same idea as 3.5, but for the DATA bitmap in block 2. Bit index 0 in the data bitmap represents absolute block DATA_REGION_BLOCK + 0 = block 4, which is where the root directory's entries live. Setting this bit means 'block 4 is taken', so future first-fit searches will correctly skip over it.

3.7 TODO 4 — Initializing the root inode

```
memset(&root_inode, 0, sizeof(root_inode));
root_inode.type = TYPE_DIRECTORY;
root_inode.links = 2;
root_inode.size = 2 * DIRENT_SIZE;
root_inode.direct[0] = ROOT_DATA_BLOCK;
root_inode.direct[1] = 0;
root_inode.direct[2] = 0;
fseek(fp, inode_offset(ROOT_INODE), SEEK_SET);
fwrite(&root_inode, sizeof(root_inode), 1, fp);
```

type = TYPE_DIRECTORY (2) because the root is a directory, not a plain file. links = 2 because two directory entries point at inode 1: "." (itself) and ".." (its own parent, since root has none). size = 2 * 64 = 128 bytes, because the root directory's data currently holds exactly two 64-byte dirent records. direct[0] = 4 (ROOT_DATA_BLOCK) says 'this directory's data lives in block 4'; direct[1] and direct[2] stay 0 because a fresh root only needs one block. inode_offset(ROOT_INODE) computes exactly where inode 1 lives inside block 3 (byte 12288, since it's the first inode), and we write the 128-byte struct there.

3.8 TODO 5 & 6 — Creating "." and ".."

```
memset(&dot, 0, sizeof(dot));
dot.inode_no = ROOT_INODE;
dot.type = TYPE_DIRECTORY;
strncpy(dot.name, ".", sizeof(dot.name) - 1);

memset(&dotdot, 0, sizeof(dotdot));
dotdot.inode_no = ROOT_INODE;
dotdot.type = TYPE_DIRECTORY;
strncpy(dotdot.name, "..", sizeof(dotdot.name) - 1);

fseek(fp, ROOT_DATA_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(&dot, sizeof(dot), 1, fp);
fwrite(&dotdot, sizeof(dotdot), 1, fp);
```

Both entries point inode_no at ROOT_INODE (1) because root has no parent to point to instead. strncpy copies the string safely: sizeof(dot.name)-1 = 58 is the MAXIMUM number of characters copied, leaving at least 1 byte of the 59-byte array guaranteed available for the terminating '\0'. Since the whole struct was memset to 0 first, and "."/".." are far shorter than 58 characters, the name field ends up correctly null-terminated with the rest of the bytes zero. We seek to block 4 (byte 16384) and write dot then dotdot back-to-back — dot lands at bytes 16384–16447 (dirent 0) and dotdot at 16448–16511 (dirent 1), matching Test Case 5 in the spec.

3.9 Closing up

```
fclose(fp);
printf("SimpleFS image created successfully: %s\n", image_name);
return 0;
}
```

fclose flushes any buffered writes to disk and releases the file handle — always important to do, otherwise some of our fwrite() calls might not actually be on disk yet. return 0 signals success to the shell (return 1 elsewhere in the program signals failure, which is the standard C convention: 0 = success, non-zero = some kind of error).

PART 4 — simplefs_adder.c Explained Line by Line

Rakib's part — but Afia must know this too

Job of this program: take one existing SimpleFS image and one file from the current folder, and register that file inside the image — allocate an inode, allocate enough data blocks, copy the bytes in, and add a directory entry for it.

4.1 Helper functions

```
void set_bit(...) { ... } // same as builder
int is_bit_set(...) { ... } // same as builder
long inode_offset(...) { ... } // same as builder
int data_bitmap_index(int absolute_block_number) {
return absolute_block_number - DATA_REGION_BLOCK;
}
```

set_bit, is_bit_set and inode_offset work exactly as explained in Part 3.1 — this program defines its own copies since it's a completely separate executable. data_bitmap_index does the REVERSE of the +4 rule from Part 1.9: given an absolute block number (e.g. 5), it tells you which bit in the data bitmap represents it (5 – 4 = bit index 1). We need this because once we've picked an absolute block via first-fit, we still have to know which specific bit to flip to 1 in the bitmap.

4.2 find_free_inode() — TODO 1

```
int find_free_inode(unsigned char *bitmap)
{
for (int i = 1; i < TOTAL_INODES; i++) {
if (!is_bit_set(bitmap, i)) {
return i + 1;
}
}
return -1;
}
```

We loop i from 1 to 31 (NOT starting at 0), because bit 0 always belongs to the root inode and is always allocated — there's no point checking it, we already know it's taken. For every other bit i, if is_bit_set returns 0/false ('not allocated'), we return i+1: remember bit index i corresponds to inode number i+1 (bit 1 -> inode 2, bit 2 -> inode 3, etc). If we scan through all 31 remaining bits and every one is already set, we fall out of the loop and return -1, meaning 'no free inode, the file system is full of files' — which main() checks and reports as an error.

4.3 find_free_data_block() — TODO 2

```
int find_free_data_block(unsigned char *bitmap)
{
for (int i = 0; i < DATA_BLOCKS; i++) {
if (!is_bit_set(bitmap, i)) {
return DATA_REGION_BLOCK + i;
}
}
return -1;
}
```

Same first-fit idea, but this time we scan ALL 60 data-block bits (i from 0 to 59), because unlike inode 1, there's no fixed bit we can skip in general — although in practice bit 0 (block 4, the root's block) will already be set from the builder, so the scan will naturally skip past it. We return DATA_REGION_BLOCK + i (i.e. 4 + i) to convert

the bitmap index back into an ABSOLUTE block number, which is what gets stored in an inode's direct[] array. Returns -1 if the disk is completely full of data.

4.4 filename_exists() — TODO 3

```
int filename_exists(FILE *image, const char *filename)
{
dirent_t entry;
for (int i = 0; i < (int)(BLOCK_SIZE / sizeof(dirent_t)); i++) {
long pos = ((long)ROOT_DATA_BLOCK * BLOCK_SIZE) + ((long)i * sizeof(dirent_t));
fseek(image, pos, SEEK_SET);
if (fread(&entry, sizeof(entry), 1, image) != 1) break;
if (entry.inode_no != 0 && strcmp(entry.name, filename) == 0) {
return 1;
}
}
return 0;
}
```

BLOCK_SIZE / sizeof(dirent_t) = 4096/64 = 64, so we check ALL 64 possible directory slots (including "." and ".." — harmless, since a real file will never be named exactly "." or ".."). For each slot i, pos computes the exact byte position of that directory entry: start of block 4, plus i entries of 64 bytes each — this is the direct application of the block-offset formula from Part 1.2, just applied to entries instead of whole blocks. We read that one dirent_t into 'entry'. entry.inode_no != 0 filters out empty/unused slots (recall 0 means 'nothing here', from Part 2.4). strcmp(...) == 0 means an exact name match. If both conditions hold, the name is already taken, so we return 1 (true/exists). If we finish the whole loop without a match, we return 0.

4.5 find_free_directory_entry() — TODO 4

```
int find_free_directory_entry(FILE *image)
{
dirent_t entry;
for (int i = 2; i < (int)(BLOCK_SIZE / sizeof(dirent_t)); i++) {
long pos = ((long)ROOT_DATA_BLOCK * BLOCK_SIZE) + ((long)i * sizeof(dirent_t));
fseek(image, pos, SEEK_SET);
if (fread(&entry, sizeof(entry), 1, image) != 1) break;
if (entry.inode_no == 0) {
return i;
}
}
return -1;
}
```

Almost identical to filename_exists, but with two key differences. First, i starts at 2, not 0 — we deliberately SKIP slots 0 and 1 because those are permanently reserved for "." and ".." and must never be overwritten. Second, we're looking for entry.inode_no == 0 (an EMPTY slot we can use for a new file), rather than a name match. Returns the first free slot INDEX (2 to 63), or -1 if the root directory is completely full (all 62 usable slots taken — though in practice we'd run out of inodes at 31 files long before we ran out of the 62 directory slots, since there are only 32 inodes total).

4.6 main() — opening and validating the image

```
image = fopen(image_name, "rb+");
if (!image) { printf("Error: file-system image not found.\n"); return 1; }
fseek(image, SUPERBLOCK_BLOCK * BLOCK_SIZE, SEEK_SET);
if (fread(&sb, sizeof(sb), 1, image) != 1) { ... return 1; }
if (sb.magic != MAGIC_NUMBER) { printf("Error: invalid SimpleFS image.\n"); ... }
```

"rb+" opens for reading AND writing in binary mode, but — unlike the builder's "wb+" — it does NOT create the file or erase it if it already exists; it fails (fopen returns NULL) if the image doesn't exist yet, which is exactly what Test Case 15 (missing image) checks for. We then read the superblock back from block 0 and compare sb.magic against MAGIC_NUMBER. This magic-number check protects us from someone accidentally pointing simplefs_adder at a random unrelated file — without it, the program might crash or corrupt data trying to interpret garbage bytes as valid metadata.

4.7 Opening the source file and checking its size

```
source = fopen(source_name, "rb");
if (!source) { printf("Error: source file not found.\n"); ... return 1; }
fseek(source, 0, SEEK_END); file_size = ftell(source); rewind(source);
if (file_size < 0) { ... }
if (file_size > MAX_FILE_SIZE) { printf("Error: file is too large for SimpleFS.\n"); ... }
```

A classic C trick for finding a file's size: seek to the very END of the file (SEEK_END with offset 0), then ftell() tells you the CURRENT cursor position — which, since we're at the end, IS the file's size in bytes. rewind(source) then resets the cursor back to the beginning so we can read the file's actual contents later. We reject anything bigger than MAX_FILE_SIZE (12288 bytes = 3 direct blocks), since SimpleFS has no indirect pointers and physically cannot store more than 3 blocks per file.

4.8 TODO 5 — Required blocks + filename length check

```
if (strlen(source_name) > 58) {
printf("Error: file name is too long for SimpleFS.\n");
fclose(source); fclose(image); return 1;
}
required_blocks = (int)((file_size + BLOCK_SIZE - 1) / BLOCK_SIZE);
```

We added the filename-length check here because the spec explicitly requires rejecting names over 58 characters (dirent.name only has 59 bytes, one of which must be the '\0' terminator — see Part 2.4). The blocks formula is the standard 'round up' trick: adding (BLOCK_SIZE – 1) before doing integer division forces any remainder to push the result up by one. Example: 5000 bytes -> (5000+4095)/4096 = 9095/4096 = 2 (integer division truncates) — matches the spec's table exactly. For file_size = 0, (0+4095)/4096 = 0, so an empty file correctly needs zero blocks without any special-case code.

4.9 Duplicate check, then reading both bitmaps

```
if (filename_exists(image, source_name)) { printf("Error: file already exists in SimpleFS.\n");
... }

fseek(image, INODE_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fread(inode_bitmap, BLOCK_SIZE, 1, image);
free_inode = find_free_inode(inode_bitmap);
if (free_inode == -1) { printf("Error: no free inode available.\n"); ... }

fseek(image, DATA_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fread(data_bitmap, BLOCK_SIZE, 1, image);
```

We call filename_exists (Part 4.4) before allocating anything, to fail fast and cheaply if the name is already taken. Then we pull BOTH full 4096-byte bitmaps off disk into local arrays in memory — every allocation decision from here on happens in memory first, and only gets written back to disk near the very end. This matters: if something fails partway through (e.g. not enough free blocks), we can just fclose() and return, and the on-disk image is left completely untouched, because we haven't written anything yet.

4.10 TODO 6 — Allocating the data blocks

```
for (int i = 0; i < required_blocks; i++) {
int block = find_free_data_block(data_bitmap);
if (block == -1) {
printf("Error: not enough free data blocks.\n");
fclose(source); fclose(image); return 1;
}
set_bit(data_bitmap, data_bitmap_index(block));
allocated_blocks[i] = block;
}
```

We loop exactly required_blocks times. Each iteration calls find_free_data_block on the IN-MEMORY bitmap (which we're about to update), so consecutive calls correctly return DIFFERENT blocks — once we set_bit for a block we just found, the next call to find_free_data_block will skip right past it since it now shows as allocated. We convert the returned absolute block number back into a bitmap index using data_bitmap_index (Part 4.1) before calling set_bit, since set_bit expects a bit INDEX, not an absolute block number — mixing these up is the exact trap warned about in Part 1.9. We remember every block we allocated in the allocated_blocks[] array so we can use them in the next two steps.

4.11 Finding a free directory slot

```
directory_entry_index = find_free_directory_entry(image);
if (directory_entry_index == -1) { printf("Error: root directory is full.\n"); ... }
```

Straightforward call into the helper from Part 4.5. Note this reads directly from the IMAGE FILE (not an in-memory copy), because the root directory's content was never loaded into memory the way the bitmaps were — we only need to know one number (the free slot index) at this point.

4.12 TODO 7 — Copying the file's actual bytes in

```
unsigned char buffer[BLOCK_SIZE];
long remaining = file_size;
for (int i = 0; i < required_blocks; i++) {
long to_read = remaining < BLOCK_SIZE ? remaining : BLOCK_SIZE;
memset(buffer, 0, BLOCK_SIZE);
if (to_read > 0) {
if (fread(buffer, 1, (size_t)to_read, source) != (size_t)to_read) { ... }
}
fseek(image, (long)allocated_blocks[i] * BLOCK_SIZE, SEEK_SET);
fwrite(buffer, BLOCK_SIZE, 1, image);
remaining -= to_read;
}
```

This is the heart of the file-copy logic. We track 'remaining' bytes still left to copy, starting at the full file_size. For each block we allocated, to_read is whichever is SMALLER: what's left (remaining) or a full block (BLOCK_SIZE) — this correctly handles the LAST block of a file, which is usually only partially full. Crucially, we memset(buffer, 0, BLOCK_SIZE) BEFORE reading, so any bytes in that last partial block that AREN'T overwritten by fread stay zero — this is exactly the spec's 'zero-fill the final block' requirement from the Common Mistakes section. We fread to_read bytes from the source file into buffer, seek to the correct absolute position of allocated_blocks[i] on disk, and fwrite the WHOLE 4096-byte buffer (data + zero padding) there. Then remaining shrinks by to_read for the next loop iteration.

4.13 TODO 8 — Building the new file's inode

```
memset(&new_inode, 0, sizeof(new_inode));
new_inode.type = TYPE_FILE;
new_inode.links = 1;
new_inode.size = (uint32_t)file_size;
for (int i = 0; i < MAX_DIRECT_BLOCKS; i++) {
```

```
new_inode.direct[i] = (i < required_blocks) ? (uint32_t)allocated_blocks[i] : 0;
}
fseek(image, inode_offset(free_inode), SEEK_SET);
fwrite(&new_inode, sizeof(new_inode), 1, image);
```

type = TYPE_FILE (1) since this is a regular file, not a directory. links = 1 because exactly one directory entry (the one we're about to create) points at it. size is the REAL byte count of the file (e.g. 5000), which is intentionally different from how many bytes are physically allocated (e.g. 2 blocks = 8192 bytes) — this matches the spec's explicit warning: 'store actual size, not allocated size'. The direct[] loop fills in allocated_blocks[i] for every block we actually used, and leaves any UNUSED direct pointer slots at 0 — e.g. a 1-block file gets direct = {block, 0, 0}. Finally we compute exactly where inode number free_inode lives (via inode_offset, Part 3.1) and write the full 128-byte struct there.

4.14 TODO 9 — Marking the inode allocated, saving both bitmaps

```
set_bit(inode_bitmap, free_inode - 1);
fseek(image, INODE_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(inode_bitmap, BLOCK_SIZE, 1, image);
fseek(image, DATA_BITMAP_BLOCK * BLOCK_SIZE, SEEK_SET);
fwrite(data_bitmap, BLOCK_SIZE, 1, image);
```

free_inode - 1 converts inode NUMBER back into a bit INDEX (the inverse of find_free_inode's i+1), so we flip the correct bit on in our in-memory copy. THEN we write BOTH bitmaps — inode bitmap and data bitmap — back to their blocks on disk. This is the first moment any of our allocation decisions actually become permanent on disk; everything before this point was just local memory changes.

4.15 TODO 10 — Writing the new directory entry

```
memset(&new_entry, 0, sizeof(new_entry));
new_entry.inode_no = (uint32_t)free_inode;
new_entry.type = TYPE_FILE;
strncpy(new_entry.name, source_name, sizeof(new_entry.name) - 1);
new_entry.name[sizeof(new_entry.name) - 1] = '\0';
{
long pos = ((long)ROOT_DATA_BLOCK * BLOCK_SIZE) + ((long)directory_entry_index *
sizeof(dirent_t));
fseek(image, pos, SEEK_SET);
fwrite(&new_entry, sizeof(new_entry), 1, image);
}
```

inode_no links the entry to the inode we just created. type marks it as a file. strncpy copies up to 58 characters of the file's name in (since we already rejected names over 58 characters back in TODO 5, this will never truncate real data) — and the explicit new_entry.name[58] = '\0' line is a defensive belt-and-suspenders guarantee that the string is ALWAYS null-terminated no matter what (strncpy technically doesn't guarantee null-termination if the source string is exactly as long as the limit, so this extra line removes any doubt). pos is computed with the same block-offset formula as everywhere else, applied to the free slot index we found in TODO 4. We then write the 64-byte entry into that exact slot in block 4.

4.16 TODO 11 — Updating the root directory's size

```
fseek(image, inode_offset(ROOT_INODE), SEEK_SET);
fread(&root_inode, sizeof(root_inode), 1, image);
root_inode.size += (uint32_t)sizeof(dirent_t);
fseek(image, inode_offset(ROOT_INODE), SEEK_SET);
fwrite(&root_inode, sizeof(root_inode), 1, image);
```

We read the ROOT's own inode back from disk (its size currently reflects however many entries existed before this call — 128 bytes for a fresh root, or more if files were already added). We add exactly sizeof(dirent_t) = 64

bytes — one more directory entry's worth — and write the inode back. This is why the spec says root size grows 128 -> 192 -> 256 -> ... as each file is added: it is literally counting how many 64-byte directory-entry slots are in active use.

4.17 Closing up

```
fclose(source); fclose(image);
printf("%s added successfully to %s\n", source_name, image_name);
return 0;
}
```

Same idea as the builder: close both open files to flush any buffered writes, print a success message, and return 0 to signal success.

PART 5 — Likely Viva Questions & Model Answers

5.1 General concept questions (either member should answer these)

What is an inode and why doesn't it store the file name?

An inode stores a file's metadata — type, size, and pointers to its data blocks — but not its name. Separating the name from the metadata (the name lives only in the directory entry) is a classic file-system design choice: it's what allows one inode to be referenced by multiple names (hard links) in real systems, even though SimpleFS itself doesn't implement links.

Why is the entire 4096-byte block reserved for the superblock/bitmaps even though the actual data is much smaller?

So that every region starts on a clean block boundary. This keeps the fseek() math simple everywhere (block_number * BLOCK_SIZE) and matches how real file systems align structures to block boundaries for simplicity and I/O efficiency, even at the cost of some wasted space.

Walk me through what happens, step by step, when we run ./simplefs_adder --input disk.img --file test1.txt.

Open and validate the image (magic number check) -> open the source file and get its size -> reject if too big or name too long -> reject if the name already exists -> load both bitmaps into memory -> find a free inode via first-fit -> compute required blocks and find that many free data blocks via first-fit -> find a free directory slot -> copy the file's bytes into the allocated blocks (zero-padding the last one) -> build and write the new inode -> mark the inode and data blocks allocated on disk -> write the new directory entry -> increase the root directory's size by 64 bytes -> close files.

What does 'first-fit' mean, and where in the code is it implemented?

First-fit means: scan the bitmap from the very start, and allocate the very first bit that is still free (0), rather than searching for the 'best' free bit by any other criteria. It's implemented in find_free_inode() and find_free_data_block() in simplefs_adder.c — both are simple linear for-loops that return as soon as they find a 0 bit.

Why do we need a magic number in the superblock?

It's a quick sanity check. When simplefs_adder opens an existing image, it reads the superblock and compares the magic field against MAGIC_NUMBER (0x53465331). If someone points the program at a random file (or a corrupted image), the magic number almost certainly won't match, and we can reject it immediately with a clear error instead of crashing or silently corrupting data.

What is the maximum file size in SimpleFS, and why?

12288 bytes. Each inode has exactly 3 direct pointers (no indirect blocks), and each block is 4096 bytes, so 3 * 4096 = 12288 is the absolute ceiling — there is physically nowhere else to point to more data.

What is the maximum number of files SimpleFS can hold?

31 regular files. There are 32 total inodes, but inode 1 is permanently reserved for the root directory, leaving 31 for user files (even though the root directory block technically has room for 62 entries).

What would happen if you forgot to zero-fill the last block of a file before writing it?

Whatever leftover bytes happened to be sitting in the buffer (uninitialized stack memory, or leftover data from a previous loop iteration) would get written to disk as garbage past the end of the real file data. That's a real bug class in actual file systems too (information leaks from stale disk contents), which is why the spec explicitly calls it out.

5.2 Builder-specific questions

Afia

Why does the root inode's links field start at 2, not 1?

Because two directory entries point at inode 1: "." (the directory referring to itself) and ".." (referring to its own parent — which, since root has no real parent, is also itself). A normal file only ever has links = 1.

Why is the root inode's initial size 128 and not 0?

size tracks how many bytes of real directory-entry data exist, not how much space is 'reserved'. A fresh root directory already contains two 64-byte entries ("." and ".."), so 2 * 64 = 128 bytes are genuinely in use from the start.

What would break if simplefs_builder didn't zero-fill the whole 262144-byte image first?

Every byte we don't explicitly set afterwards — the padding in the superblock block, the unused inode-bitmap/data-bitmap bytes beyond bit 0, all 31 unused inode slots in the inode table, and every unused directory-entry slot in the root block — would contain random leftover data from the filesystem instead of clean zeros, breaking every 'must be zero' requirement in the spec and likely breaking find_free_inode/find_free_data_block, which rely on 0 meaning 'free'.

Why are set_bit and inode_offset defined separately in both simplefs_builder.c and simplefs_adder.c instead of just once?

Because they're two entirely separate C programs, each compiled into its own independent executable. simplefs.h only DECLARES these functions (their signatures); each .c file must provide its own DEFINITION (actual function body) so each program links and runs correctly on its own.

5.3 Adder-specific questions

Rakib

Why do we check filename length AND filename duplication BEFORE allocating any inode or data block?

To fail fast and cheaply. If the name is invalid or already taken, there's no point doing the more expensive work of scanning bitmaps and preparing an allocation, and — more importantly — it means we never touch the disk at all for a request we're about to reject, so the image stays completely untouched on any rejected operation.

Trace through required_blocks = (file_size + BLOCK_SIZE - 1) / BLOCK_SIZE for file_size = 8193.

(8193 + 4095) / 4096 = 12288 / 4096 = 3. This matches the spec's table exactly: an 8193-byte file needs 3 blocks, because 2 full blocks (8192 bytes) aren't quite enough — the 1 extra byte spills into a 3rd block.

Why does find_free_data_block get called inside a loop when allocating multiple blocks, rather than just once?

Because we might need more than one block per file (up to 3). Each call to find_free_data_block operates on the CURRENT state of the in-memory data_bitmap, which we update with set_bit immediately after each allocation — so the next call in the loop correctly skips the block we just claimed and finds the next free one.

What is the difference between data_bitmap_index() and DATA_REGION_BLOCK + i?

They're exact inverses of each other. DATA_REGION_BLOCK + i converts a bitmap INDEX into an absolute BLOCK number (used inside find_free_data_block, and when storing values into inode.direct[]). data_bitmap_index(block) converts an absolute BLOCK number back into a bitmap INDEX (needed right

before calling set_bit, since set_bit expects an index, not a block number).

If simplefs_adder crashes or is killed halfway through adding a file, could the disk image end up corrupted?

It's possible, but the code is ordered to minimize the risk: nothing is written to disk until AFTER all validation and all allocation decisions are made in memory. The riskiest window is between writing the new inode/bitmaps/directory entry and updating the root inode's size — if we're interrupted in that narrow window, the image could be briefly inconsistent. Full crash-safety (journaling) is explicitly listed in the spec as something SimpleFS does NOT implement.

Why do we still read root_inode fresh from disk in TODO 11 instead of reusing a value from earlier in main()?

Because main() never actually loaded the root inode into memory before this point — only the two bitmaps were cached locally. Re-reading it here gets its current, correct size (which may have already grown from previous simplefs_adder runs on this same image) before we add 64 more bytes to it.

5.4 Tricky 'what if' questions examiners like to ask

BOTH MEMBERS

What if I add a 4096-byte file exactly?

required_blocks = (4096+4095)/4096 = 8191/4096 = 1 (integer division truncates down). Exactly one full block is used, with zero wasted bytes — no partial/zero-padded block needed at all.

What if the disk is completely full of files (31 files added) and I try to add one more?

find_free_inode() scans all 31 non-root bits, finds every one set, and returns -1. main() checks free_inode == -1 and prints 'Error: no free inode available.' and exits with code 1 — no crash.

What if two files are added with the same content but different names?

That's completely fine — filename_exists() only checks the NAME field of existing directory entries, not the file content. Both would get separate inodes and separate (possibly overlapping in content, but not in block ownership) data blocks.

Why does the spec say Direct pointer value of 0 means unused — could a real block legitimately be numbered 0?

No — block 0 always holds the superblock and is never part of the data region (the data region starts at block 4). So 0 can safely be reused as a sentinel meaning 'no block here', the same way inode_no = 0 safely means 'empty directory slot' since inode numbers start at 1.

What's the difference between inode.size and how many bytes are actually allocated on disk for that file?

inode.size is the file's REAL byte count (e.g. 5000). Allocated space is always a whole number of 4096-byte blocks (e.g. 2 blocks = 8192 bytes for that same file). The difference (8192 – 5000 = 3192 bytes here) is internal fragmentation — wasted space inside the last allocated block, padded with zeros.

If I open the image with a hex editor and manually flip a bitmap bit, what breaks?

If you flip an allocated bit to free without also clearing that block/inode's actual data, the next first-fit allocation could hand that same block out again to a second file, silently corrupting the first file's data — this is exactly why the two bitmaps and the real inode/data contents must always be kept in sync.

PART 6 — One-Page Formula Cheat Sheet

What you want	Formula
Byte offset of block N	`N * BLOCK_SIZE  (N * 4096)`
Byte offset of inode number n	`12288 + (n - 1) * 128   [inode_offset()]`
Byte offset of dirent slot i (in root block)	`16384 + i * 64`
Absolute block from data-bitmap index i	`4 + i   (DATA_REGION_BLOCK + i)`
Data-bitmap index from absolute block b	`b - 4   (data_bitmap_index())`
Set bit 'index' in a bitmap	`bitmap[index/8] |= (1 << (index%8))`
Check bit 'index' in a bitmap	`bitmap[index/8] & (1 << (index%8))`
Blocks required for a file of size s bytes	`(s + 4095) / 4096   (integer division)`
Max file size	`3 * 4096 = 12288 bytes`
Max file name length	`58 characters (+1 for '\0' = 59-byte field)`
Max regular files	`31 (32 inodes - 1 for root)`

Good luck in the viva — if you can explain, in your own words, why each formula above looks the way it does (not just recite it), you'll be able to answer almost anything the instructor asks.